# Truth Table Generator using Python

Dinanath Padhya*, Krishna Acharya†
*THA078BEI010 †THA078BEI020
Department of Electronics, Communication and Information Engineering
Thapathali Campus, IOE, Tribhuvan University, Nepal

***Abstract*—This lab focused on designing and implementing a Python-based truth table generator that accepts logical expressions written using standard propositional logic symbols. The objective was to evaluate and display the truth value of logical statements under all possible combinations of boolean variable assignments. The logical operators supported include negation (¬), conjunction (∧), disjunction (∨), implication (→), and bi-implication (↔). The project involved processing the input expression, extracting variables, converting logic syntax into Python-compatible format, generating truth combinations, evaluating the expression, and finally printing a structured truth table. This lab served as an introduction to symbolic logic computation in code, reinforcing key concepts from propositional logic through a practical implementation approach.**

***Index Terms*—Truth Table, Logic Expression, Python Programming, Boolean Logic, Expression Evaluation**

## I. Introduction

Truth tables are a fundamental tool in logic, artificial intelligence, and digital systems. They list all possible combinations of truth values for a set of variables and indicate whether a logical expression evaluates to true or false under each of those combinations. In this lab, the task was to build a truth table generator using Python, which evaluates a propositional logic formula written with human-readable symbols and automatically produces the correct truth table for any valid expression. The motivation behind this lab was not only to automate the process of generating truth tables but also to understand the internal mechanics of evaluating boolean expressions at a deeper level. Implementing the evaluation logic manually ensured familiarity with both syntax manipulation and boolean reasoning, making the lab a useful exercise in both programming and logical thinking.

## II. Objective

The main objective of this lab was to implement a Python program that can read a logical expression written using standard logical operators and output the corresponding truth table. This includes handling expressions with varying numbers of variables, processing logical symbols that are not natively recognized by Python, and correctly evaluating all possible combinations of truth values. The program had to support core logical operations, including negation, conjunction, disjunction, implication, and equivalence, with the ability to handle nested expressions. Beyond just generating the table, the goal was to ensure correctness, flexibility, and clarity in the output, allowing it to be used as an educational or verification tool for learning and teaching propositional logic.

## III. Methodology

The implementation began with defining a function to extract all unique variables from the input logical expression. Since the variables are assumed to be single uppercase letters (such as A, B, or C), this task was accomplished by scanning each character of the input and adding every alphabetic character to a set. After the set was formed, it was converted into a sorted list to maintain order during output formatting.

Next, the logical expression had to be preprocessed to replace all human-readable logical symbols with Python equivalents. The negation symbol (¬) was replaced with the Python keyword `not`, conjunction (∧) with `and`, and disjunction (∨) with `or`. These replacements could be handled with simple string substitutions since the operators map directly to Python's boolean syntax. However, the implication (→) and equivalence (↔) operators required more care because they are not built into Python and must be expanded into equivalent expressions.

To handle implication, the expression `A -> B` was transformed into `not A or B`, which reflects the correct logical behavior. Similarly, equivalence expressions like `A <-> B` were expanded into `(A and B) or (not A and not B)`. These transformations were handled using regular expressions, allowing the code to identify patterns involving operators and operands, even in nested expressions. To ensure all such occurrences were captured and correctly replaced, the code repeatedly applied regular expression substitutions until the expression was free of any remaining implication or equivalence symbols.

Once the expression was fully converted into Python syntax, the next step was to generate all possible combinations of truth values for the variables involved. This was achieved using Python's `itertools.product` function, which can generate Cartesian products over the values `[False, True]` repeated for the number of variables. Each resulting tuple represented a unique truth assignment for the variables.

For each combination of truth values, the variable names in the expression were replaced with their corresponding boolean values as strings. The result was a fully Python-compatible boolean expression containing only the words `True`, `False`, and Python logical operators. This expression was then evaluated using Python's `eval()` function, which returned the truth value for that particular combination. If any syntax error occurred during evaluation, it was caught and reported, although proper expression formatting generally prevented such issues.

The main code for evaluating a row in the truth table looked like the following:

```python
def eval_expr(expr, value_map):
    eval_expr = expr
    for var, val in value_map.items():
        eval_expr =
            eval_expr.replace(var,
            str(val))
    return eval(eval_expr)
```

Listing 1. Evaluating a truth table row

Finally, the program formatted and printed a truth table. The header row displayed all the variables in order, followed by a `Result` column. Each row of the table corresponded to one combination of truth values, followed by the evaluated result. Boolean values were displayed as 0 and 1 for readability.

## IV. Results

The program was tested on five different logical expressions of varying complexity. These expressions included basic combinations such as `not A or B`, as well as more complex ones like `(A and B) <-> (C or not A)`. The output for each expression included a complete truth table, displaying all possible combinations of variable values and the resulting evaluation of the expression. For example, when evaluating the expression `(A and B) <-> (C or not A)`, the table correctly showed that the expression is true for input combinations such as A = 0, B = 0, C = 1, and false for combinations like A = 1, B = 0, C = 0. The total number of rows in the table was determined by the number of unique variables in the expression. For three variables, the table contained eight rows, covering all possible 0/1 input combinations. The following logical expressions were evaluated using the implemented Python program. Their corresponding truth tables were generated and verified manually.

*Expression: `not A or B`*

| A | B | Result |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

*Expression: `A and B -> C`*

| A | B | C | Result |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

*Expression: `A <-> B`*

| A | B | Result |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

*Expression: `A or not B and C`*

| A | B | C | Result |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

*Expression: `(A and B) <-> (C or not A)`*

| A | B | C | Result |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

These tables confirm the correct functioning of the Python truth table generator. The output matches the expected values for all possible input combinations and logical constructs.

## V. Conclusion

This lab provided a hands-on opportunity to implement a symbolic logic evaluator and truth table generator using Python. By writing the logic manually rather than relying on a library it was possible to deeply understand the process of variable extraction, syntax transformation, and boolean evaluation. The program successfully supported all required logical operators and was able to process both simple and compound expressions without error. Through this lab, the connection between symbolic logic and its computational representation became clear. It also showed how tools like regular expressions, `eval()`, and `itertools` can be used together to solve real logic problems.